HDOC Day-49

Pivot Integer Using Pointers in C

Problem Statement

Read a positive integer n and pass its address to pivot_integer(). Find an integer x between 1 and n such that the inclusive sum 1 + ... + x equals x + ... + n. Display x or -1 when no pivot exists.

Algorithm

1. Begin and read an integer n. Reject zero, negative, or non-integer input and request another value.
2. Call pivot_integer(&n), passing the address of n to the function.
3. Calculate the total sum from 1 to n as total = n*(n+1)/2 using long long arithmetic.
4. Initialize left = 0. For each candidate x from 1 to n, add x to left (sum 1 through x).
5. Calculate right = total - left + x (sum x through n, including x).
6. If left equals right, display x as the pivot integer and return from the function.
7. If no candidate satisfies the condition, display Pivot integer: -1. Stop.

Test Case Table

TC	Input (in order)	Expected output	Purpose
1	8	Pivot integer: 6	Professor example: pivot exists
2	1	Pivot integer: 1	Single integer; boundary case
3	4	Pivot integer: -1	Professor example: no pivot
4	49	Pivot integer: 35	Another valid pivot
5	2	Pivot integer: -1	Small input with no pivot
6	0, then 8	n must be positive. Then Pivot integer: 6	Invalid zero; retry
7	-3, then 1	n must be positive. Then Pivot integer: 1	Invalid negative; retry
8	abc, then 8	Invalid input. Enter an integer. Then Pivot integer: 6	Non-integer; retry

For Test Cases 6-8, enter the values in the stated order during the same execution. The program prompts again after invalid input.

C Program

```
#include <stdio.h>

void pivot_integer(const int *n)
{
    long long total = (long long)(*n) * (*n + 1LL) / 2;
    long long left = 0;
    int x;

    for (x = 1; x <= *n; x++)
    {
        left += x;
```

```
        /* x appears in both inclusive sums. */
        long long right = total - left + x;

        if (left == right)
        {
            printf("Pivot integer: %d\n", x);
            return;
        }
    }
    printf("Pivot integer: -1\n");
}

int main(void)
{
    int n, status, ch;

    while (1)
    {
        printf("Enter a positive integer n: ");
        status = scanf("%d", &n);

        if (status == EOF)
        {
            printf("No input provided.\n");
            return 1;
        }
        if (status != 1)
        {
            printf("Invalid input. Enter an integer.\n");
            while ((ch = getchar()) != '\n' && ch != EOF) { }
            continue;
        }
        if (n <= 0)
        {
            printf("n must be positive.\n");
            continue;
        }
        break;
    }

    pivot_integer(&n);
    return 0;
}
```

Compilation and Execution

```
gcc hdoc49.c -o hdoc49
./hdoc49
```

Compilation and Execution - Screenshot



Test Case 1 - Screenshot

Input: 8 | Expected: Pivot integer: 6

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc49
Enter a positive integer n: 8
Pivot integer: 6
```

Test Case 2 - Screenshot

Input: 1 | Expected: Pivot integer: 1

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc49
Enter a positive integer n: 1
Pivot integer: 1
```

Test Case 3 - Screenshot

Input: 4 | Expected: Pivot integer: -1

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc49
Enter a positive integer n: 4
Pivot integer: -1
```

Test Case 4 - Screenshot

Input: 49 | Expected: Pivot integer: 35

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc49
Enter a positive integer n: 49
Pivot integer: 35
```

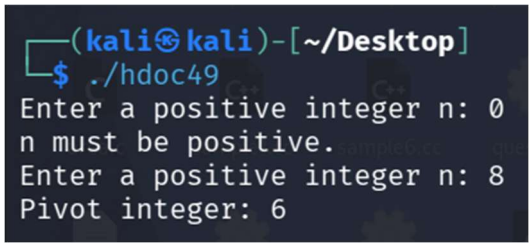
Test Case 5 - Screenshot

Input: 2 | Expected: Pivot integer: -1

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc49
Enter a positive integer n: 2
Pivot integer: -1
```

Test Case 6 - Screenshot

Input: 0, then 8 | Expected: n must be positive. Then Pivot integer: 6



Test Case 7 - Screenshot

Input: -3, then 1 | Expected: n must be positive. Then Pivot integer: 1



Test Case 8

Input: abc, then 8 | Expected: Invalid input. Enter an integer. Then Pivot integer: 6

